Lesson 2 Teacher's Script

Programming Basics I: Sequences, Events & the Game Loop

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back, everyone! Last week we talked about internet safety, what game development actually involves, and looked at a bunch of different engines. Show of hands - who got their engine installed and tested since last class?" [Pause, scan the room] "If you didn't finish, that's okay - we've got time at the start of lab today, and remember GDevelop and Scratch work straight in a browser with zero install if you're in a pinch."

[Demo cue: have your own demo laptop open with Scratch ready]

"Today we're doing our first real programming lesson. Everything from here forward builds on itself, so today we're also starting the actual game project you'll keep working on all semester."

[0:05-0:20] Sequences

"Let's start with the simplest idea in programming: a sequence. A sequence is just a list of instructions that happen one after another, in order. Think about your morning routine - wake up, get dressed, eat breakfast. If you get dressed before you wake up, that doesn't work! Order matters."

[Demo cue: switch to Scratch, open a blank project]

"Watch what happens when I stack these blocks: move 10 steps, then turn 15 degrees, then say 'Hello!' " [Click blocks one at a time, then run the stack] "See how it does them top to bottom, one after another? That's a sequence. Every single line of code in every engine we use this year - GDScript in Godot, C# in Unity, GML in GameMaker - all of it runs as sequences like this at its core."

[Discussion question] "What's something else in your daily life that has to happen in a specific order to work? Turn to a neighbor and share one example."

[Timing checkpoint: wrap up by 0:18-0:20]

[0:20-0:35] Events

"Now here's the problem with plain sequences: they just run once, top to bottom, and stop. But games aren't like that - games wait around and respond to what you do. That's called an event. An event is code that says 'when this happens, do that.'"

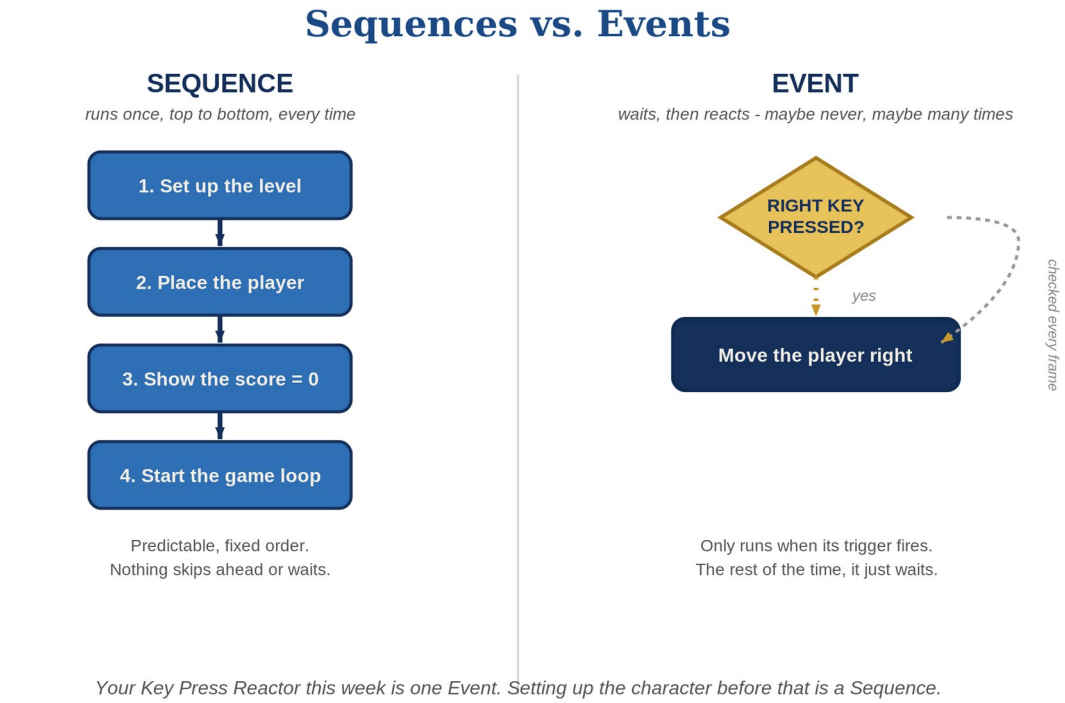
[Demo cue: same Scratch project]

"In Scratch, events start with hat-shaped blocks like 'when green flag clicked' or 'when key pressed.' " [Drag out "when right arrow key pressed" and attach "move 10 steps"] "Watch - nothing happens until I press the right arrow key." [Press right arrow several times] "Every time I press it, that whole sequence underneath runs again from the top - the character moves another 10 steps to the right. Same block as before, just triggered by a key now instead of running once at the start."

"This is the shift from 'a list of steps that happens once' to 'a list of steps that can happen over and over, whenever something triggers it.' Every engine we use has some version of this - Godot has signals and _input(), Unity has Keyboard.current key checks, GameMaker has the Key Pressed event, and so on. Different names, same idea."

[Discussion question] "What other events can you think of in games you've played?" (Examples to draw out if needed: pressing a button, an enemy touching you, a timer running out, opening a chest.)

[Display cue: put the Sequences vs. Events diagram up (slide or printed) as a wrap-up visual for both of the last two sections]



Sequences vs. Events - the two ideas covered in the last 30 minutes, side by side.

[Timing checkpoint: wrap by 0:35]

[0:35-0:50] The Game Loop

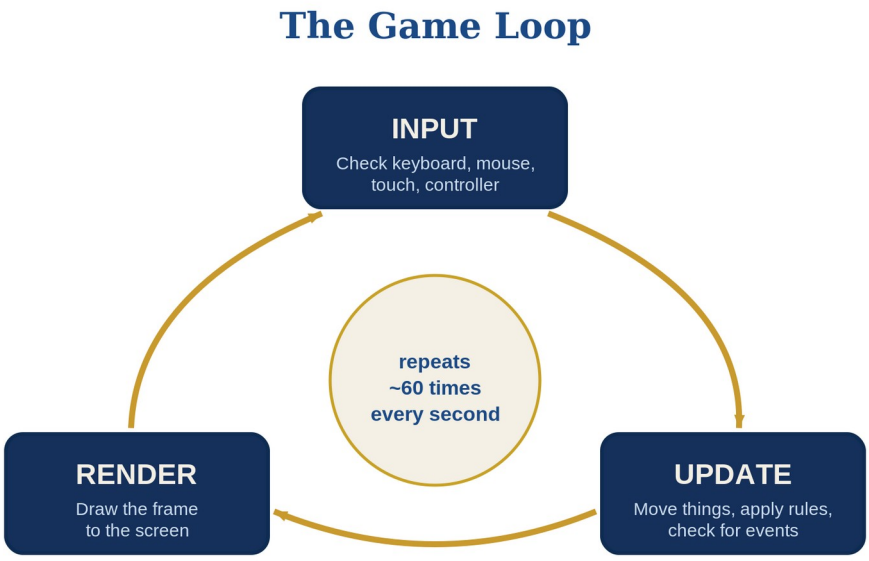
"Last big idea: the game loop. Here's a question - how does a game character keep moving smoothly while you hold down a key, instead of just twitching once?" [Pause for guesses]

"It's because the engine isn't just waiting for one event and stopping - it's constantly asking, many times every second, usually 30 to 60 times a second: 'what's the current input? what needs to move? what needs to be redrawn?' That loop of check-update-draw, check-update-

draw, happening dozens of times a second, is called the game loop. You don't usually write the loop yourself - the engine runs it for you - but almost everything you build this year runs inside it."

[Optional visual: flip-book analogy] "It's like a flip book - each page is slightly different, and flipping through them fast makes it look like smooth motion. Each 'page' is one pass through the game loop."

[Display cue: put the Game Loop diagram up (slide or printed) - trace Input -> Update -> Render -> back to Input with your finger as you talk through it again]



Every engine you use runs some version of this loop under the hood - it's why your character moves smoothly instead of jumping in one big step.

The Game Loop - Input, Update, and Render, repeating dozens of times per second.

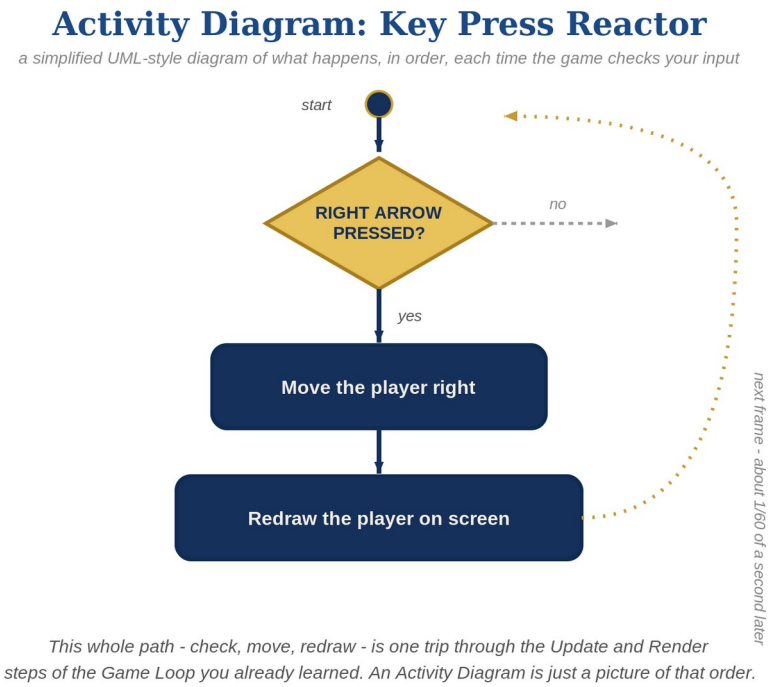
[Timing checkpoint: wrap by 0:48-0:50]

[Bonus, only if time allows] Thinking in Diagrams

This section is optional filler, not part of the timed agenda - skip it entirely if you're at time, or hold it for the start of next lesson. It's a light, no-pressure preview, not new required content.

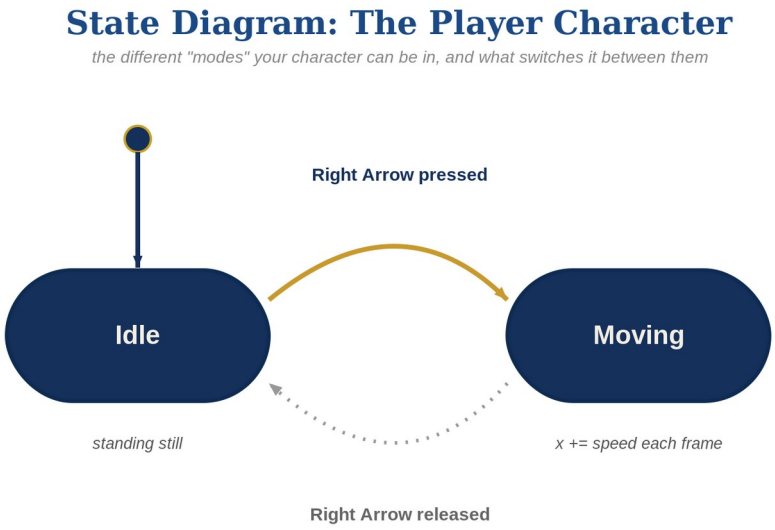
"One more thing, if we've got a few minutes: you just described the game loop out loud. Professional developers often draw it instead, using a couple of standard diagram styles. You don't need to memorize these - just start recognizing them when you see them."

"An Activity Diagram is basically a flowchart - it shows the order things happen in. Here's your Key Press Reactor as one: check the key, and if it's pressed, move the player, then redraw. Same thing you just described, as a picture."



Activity Diagram - the Key Press Reactor's check/move/redraw flow.

"A State Diagram shows the different 'modes' something can be in, and what switches it between them. Your player character has at least two: Idle and Moving. Pressing the key switches it one way, releasing switches it back."



A game character can usually only be in one state at a time. Level 2 of this week's exercise really has 5 states like this - Idle plus one Moving state per direction.

State Diagram - the player character's Idle and Moving states.

"We'll come back to a third kind - Class Diagrams - later in the year once you're writing more object-oriented code. For now, just knowing these two exist, and roughly what they're for, is enough."

[0:50-1:00] Today's Exercise & Handouts

"For lab today, you're building what I'm calling a 'Key Press Reactor' - an object that responds when you press a key. This is also the very first piece of your Semester 1 project, so save it somewhere you'll remember and give the project a name you'll recognize, like [YourName]Game."

"Everyone's getting a handout specific to their engine - the steps look a little different depending on what you're using, but the goal is the same for everybody: Level 1 is one key doing one thing. Level 2 is all four arrow keys moving your character around. Level 3 is adding some polish - animation, sound, whatever you want to add."

[Hand out engine-specific handouts]

"You're also getting your real Game Design Document today - the one your Lesson 1 sneak peek was a preview of. Find the 'Lesson 2 Goals' checklist inside it, under Levels / Structure - it's the same three levels from your handout, just in one place so you can check them off as you go. No need to fill in the rest of the document today; that can wait until you've got a working exercise to build on."

[Hand out Lesson_02_GDD.docx - one per student]

"Questions before we start lab? ... Let's go build something."

Lab Time Notes

- Circulate by engine cluster, starting with whichever engine has the most students this week.
- Common early snag across every engine: an event block/node that isn't actually connected to the rest of the logic won't run. If a student says "nothing happens," check connections first.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout, not given new unrelated work.
- At 1:50, call time and do a 2-minute share-out: ask 2-3 students to show what they built, across different engines if possible.